Spring Boot Interview Ques-

Q1. What is Autowiring?

Autowiring in Spring is the automatic dependency injection mechanism where Spring IOC container automatically injects required beans into dependent classes.

Q2. What is difference bet @Component and @Bean?

In Spring Framework, both `@Component` and `@Bean` are used to create **Spring Beans**, but they are used in different ways and different situations.

Core Difference -

`@Component`	`@Bean`
Used on a **class**	Used on a **method**
Spring automatically detects it using component scanning	You manually define bean creation
Best for your own classes	Best for third-party/external classes
Declarative	Explicit/manual

- `@Component` is used for automatic bean creation through component scanning, while `@Bean` is used for manual bean creation inside a configuration class, usually when we need more control or when working with third-party classes.

Q3. Difference between spring and spring boot?

Spring Framework is a complete enterprise Java framework requiring more manual configuration, while Spring Boot is an extension of Spring that simplifies development using auto-configuration, starter dependencies, and embedded servers.

Q4. Which components does @SpringBootApplication combines?

Single annotation combines:

- `@Configuration`
- `@EnableAutoConfiguration`
- `@ComponentScan`

Q5. What is the purpose of @EnableAutoConfiguration?

It enables Spring Boot's auto-configuration mechanism, where Spring Boot automatically configures application beans based on classpath dependencies and existing configurations.

Q6. What is the purpose of @ComponentScan?

`@ComponentScan` tells Spring where to scan and discover component classes so they can be registered as beans inside the IOC container.

```
@ComponentScan( basePackages = {
"com.example",
"com.other.service"
})
```

Q7. What is the purpose of @Configuration on main class?

It means that main class can be considered as configuration class i.e. you can declare beans inside this class.

Q8. Who is traffic controller in spring boot?

`DispatcherServlet` is called the traffic controller or front controller in Spring MVC because it handles all incoming requests and dispatches them to appropriate controllers.

Q9. What is the purpose of @RestController?

`@RestController` is used to create REST APIs in Spring Boot. It combines `@Controller` and `@ResponseBody`, so methods return data directly as HTTP response instead of returning view pages.

REST - Representational State Transfer Application Programming Interface.

Q10. What are Bean Scopes?

Bean scopes define the lifecycle and visibility of Spring beans, determining how many instances of a bean are created and shared within the application.

Different bean scopes are –

1. Singleton
2. Prototype
3. Request
4. Session

1. Singleton - Only ONE bean object created for whole application.
2. Prototype - Creates NEW object every time bean requested.
3. Request - One bean per HTTP request.
4. Session - One bean per user session.

Q11. What is spring boot actuator?

Spring Boot Actuator is a production-ready feature that provides monitoring and management endpoints for observing application health, metrics, beans, environment properties, and runtime behavior.

Q12. Which application property is used to change the default port?

Use **`server.port`** **in** **`application.properties`** **or** **`application.yml`** to change the default Spring Boot server port.

Q13. What is the main purpose of spring profiles?

Spring Profiles are used to separate and manage environment-specific configurations so that different beans and properties can be loaded for different environments like dev, test, and production.

Example

application-dev.properties -

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost/devdb
```

application-prod.properties -

```
server.port=8080
spring.datasource.url=jdbc:mysql://prod-server/proddb
```

Activate Profile

In `application.properties`:

```
spring.profiles.active=dev
```

Now Spring loads:

- `application.properties`
- `application-dev.properties`

Using `@Profile`

You can create beans for specific environments.

Example

```
@Configuration
@Profile("dev")
public class DevConfig {
}
```

This configuration loads only when:

```
spring.profiles.active=dev
```

Multiple Profiles -

```
spring.profiles.active=dev,test
```

Q14. Can we connect multiple databases with you spring boot application?

Yes, Spring Boot supports multiple database connections by configuring multiple DataSource, EntityManagerFactory, and TransactionManager beans separately.

Q15. What is context path ? how do you define it?

Context path is the base URI prefix of a web application that is added before all endpoint URLs.

Example -

```
server.port=8081
server.servlet.context-path=/company
```

Application URL:

```
http://localhost:8081/company
```

Q16. Can you disable a particular class from spring boot application?

Yes, a class can be disabled in Spring Boot.

@SpringBootApplication(**exclude = {DataSourceAutoConfiguration.class}**)
public class MyApplication {}

Q17. What is spring boot's default serialization?

Spring Boot uses Jackson library by default for serialization and deserialization of Java objects to/from JSON using `ObjectMapper`

Flow Internally -

```
Java Object
   ↓
ObjectMapper (Jackson)
   ↓
JSON Response
```

For request body:

```
JSON Request
   ↓
ObjectMapper
   ↓
Java Object
```

This is called:

- Serialization → Object to JSON
- Deserialization → JSON to Object

Q18. What does the @GeneratedValue annotation do?

`@GeneratedValue` is used in JPA to automatically generate primary key values for entities. `@GeneratedValue` only works on primary key fields.

EG- @GeneratedValue(strategy = GenerationType.IDENTITY)

Types – 1. IDENTITY

2. SEQUENCE

3. TABLE

4. AUTO (Default strategy)

Q19. What is relationship betwen Spring Data JPA and JPA?

JPA is a specification for ORM, while Spring Data JPA is a Spring framework built on top of JPA that simplifies database operations by reducing boilerplate code.

Q20. What is the purpose of LOMBOK in s[ring boot project?

Lombok is a Java library that reduces boilerplate code in Spring Boot projects by automatically generating methods like getters, setters, constructors, and builders using annotations.

Lombok works using:

- Annotation processing
- Compile-time code generation

Generated methods are NOT physically visible in source code but exist in compiled `.class` files.

`@Data` internally adds:

- Getter
- Setter
- toString
- equals/hashCode
- RequiredArgsConstructor

Q21. Why do some projects avoid **`@Data`** on entities?

- `equals()` and `hashCode()` may create issues with Hibernate proxies
- `toString()` may trigger lazy loading
- Better to use selective annotations

Q22. What is the purpose of pagination in spring data jpa?

Pagination in Spring Data JPA is used to retrieve large datasets in smaller chunks, improving performance, memory efficiency, and response time.

Important Interfaces-

Interface	Purpose
`Pageable`	Pagination information
`Page`	Paginated result
`Slice`	Lightweight pagination

```
Example - Page<Employee> page =
employeeRepository.findAll(
    PageRequest.of(0,10)
);
```

Page 0 = first page.

10 = records per page.

Q23. What is the purpose of Interceptors in spring boot appication?

Interceptors in Spring Boot are used to intercept HTTP requests and responses for cross-cutting concerns like logging, authentication, authorization, and request monitoring before or after controller execution.

Request Flow-

```
Client Request
      ↓
Interceptor
      ↓
Controller
      ↓
Interceptor
      ↓
Response
```

Interceptor acts like a checkpoint.

Creating an Interceptor

Step 1: Implement **`HandlerInterceptor`**

```
@Component
public class MyInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler) {

        System.out.println("Before Controller");

        return true;
    }

    @Override
    public void postHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            ModelAndView modelAndView) {

        System.out.println("After Controller");
    }

    @Override
    public void afterCompletion(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            Exception ex) {

        System.out.println("After Response Completion");
    }
}
```

Step 2: Register Interceptor

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Autowired
    private MyInterceptor interceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {

        registry.addInterceptor(interceptor);
    }
}
```

Important Methods

Method	Purpose
`preHandle()`	Before controller execution

Method	Purpose
`postHandle()`	After controller, before response
`afterCompletion()`	After complete request processing

Interceptor vs Filter

Filter	Interceptor
Part of Servlet API	Part of Spring MVC
Works before DispatcherServlet	Works after DispatcherServlet
Works on all requests	Mainly controller requests
Used for low-level tasks	Used for application logic

Real-World Uses

- JWT validation
- API logging
- Request timing
- Audit tracking
- Language selection
- Rate limiting

Q24. What is the difference between Authentication and Authorization?

Authentication identifies the user, while authorization determines what resources or operations that authenticated user is permitted to access.

In Spring Security Flow

```
Login Request
     ↓
Authentication
     ↓
User Verified
     ↓
Authorization
     ↓
Access Granted/Denied
```

Q25. Difference between stateless and statefull in spring boot?

In a stateful Spring Boot application, the server maintains client session data between requests, whereas in a stateless application, each request is independent and contains all required information, typically using JWT tokens.

Real-World Usage

Stateful -

Used in:

- Traditional web apps
- Banking portals
- JSP/Servlet apps

Stateless -

Used in:

- REST APIs
- Microservices
- Mobile backends
- Modern distributed systems

Q26. Difference Between PUT and PATCH

PUT	PATCH
Full update	Partial update
Replaces object	Updates specific fields

---------- **Spring boot with DB below some Questions** ----------

Q27. What is Hibernate?

It is an ORM framework. Maps Java objects to database tables.

Without Hibernate:

```
Connection conn;
PreparedStatement ps;
ResultSet rs;
```

With Hibernate:

```
employeeRepository.save(employee);
```

Q28. What is ORM?

Object Relational Mapping.

Employee class

mapped to

employee table

Q29. What is Entity?

```
@Entity
@Table(name="employee")
public class Employee {

   @Id
   private Long id;

   private String name;
}
```

@Id - Primary key.

```
@Id
private Long id;
```

@GeneratedValue - Auto-generates IDs.

```
@Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
private Long id;
```

Generation Strategies - IDENTITY

@GeneratedValue(strategy = GenerationType.IDENTITY)

Database generates. - MySQL AUTO_INCREMENT.

SEQUENCE - @GeneratedValue(strategy = GenerationType.SEQUENCE)

Mostly Oracle/Postgres.

AUTO - Hibernate chooses.

Q30. Hibernate Relationships

OneToOne

```
@OneToOne
private Passport passport;
```

Person → Passport

OneToMany

```
@OneToMany(mappedBy="department")
private List<Employee> employees;
```

Department → Employees

ManyToOne

```
@ManyToOne
@JoinColumn(name="department_id")
private Department department;
```

Employee → Department

ManyToMany

```
@ManyToMany
@JoinTable(
 name="student_course"
)
private List<Course> courses;
```

Q31. What is mappedBy?

`mappedBy` indicates that the relationship is managed by another entity's field and prevents Hibernate from creating an additional join table.

Q32: Does mappedBy create a column?

No. The column is created by the owning side (`@ManyToOne` with `@JoinColumn`).

Q33: What happens if mappedBy points to a wrong field name?

Example:

```
@OneToMany(mappedBy = "dept")
```

But `Employee` has:

```
private Department department;
```

Application startup will fail with a mapping exception because Hibernate cannot find the field `dept`.

Q34. Fetch Types

EAGER

Loads immediately.

```
@OneToMany(fetch=FetchType.EAGER)
```

LAZY

Loads when needed.

```
@OneToMany(fetch=FetchType.LAZY)
```

Preferred in production.

Q35. What is Spring Data JPA?

Abstraction on top of Hibernate.

```
Spring Data JPA
      ↓
Hibernate
      ↓
Database
```

JpaRepository

```
public interface EmployeeRepository
extends JpaRepository<Employee,Long> {
}
```

Provides:

```
save()
findById()
findAll()
delete()
```

Custom Finder Method

```
List<Employee>
findByDepartmentId(Long id);
```

Generated automatically.

JPQL - Java Persistence Query Language

Works with Entity names.

```
@Query(
"select e from Employee e where e.salary > :salary"
)
List<Employee> getEmployees(double salary);
```

Native Query

Works with SQL.

```
@Query(
value =
"select * from employee",
nativeQuery = true
)
List<Employee> findAllEmployees();
```

Q36. N+1 Problem

Bad:

```
departmentRepository.findAll();
```

Then for every department:

```
department.getEmployees();
```

100 departments = 101 queries.

Solution:

```
JOIN FETCH
SELECT d
FROM Department d
JOIN FETCH d.employees
```